

Remote Procedure Call in Java

Advanced Computer Programming

Prof. Raffaele Della Corte

GRPC



- grpc.io è un middleware RPC **universale** ad **alte prestazioni** e **open source**
- Può essere eseguito in qualsiasi ambiente
  - Framework **multilinguaggio** e **multipiattaforma**
- Principali scenari di utilizzo
  - Collegare **microservizi** poliglotti che utilizzano lo stile di comunicazione richiesta-risposta
  - Connettere **dispositivi mobili** e browser ai servizi di backend
  - Generare **librerie client efficienti**
- Sviluppato da **Google** e ora un progetto *Cloud Native Computing Foundation (CNCF)*
- **Utilizzato da molte aziende** e in molti sistemi distribuiti
  - Ad esempio, Google, Dropbox, Netflix, Square, etcd, CockroachDB

GRPC per Java: requisiti



- Installare il compilatore **protoc**
  - **Mac OSX, Linux**
    - Scaricare la versione per Linux o MacOSX di protoc da **https://github.com/protocolbuffers/protobuf/releases**
      - Controlla versioni protoc-XX.X_linux-x86_64.zip, protoc-XX.X-linux-x86_32.zip, protoc-XX.X_osx-universal_binary.zip
    - Decomprimere il file come di seguito (sostituire il file con il nome reale!)

sudo unzip -o protoc-XX.X_linux-x86_64.zip -d /usr/local bin/protoc

sudo unzip -o XXX_linux-x86_64.zip -d /usr/local 'include/*'

  - **Windows**
    - Scaricare la versione per Windows di protoc da **https://github.com/protocolbuffers/protobuf/releases**
      - **protoc-XX.X-win64, protoc-XX.X-win32**
    - Decomprimere il file e impostare la variabile di sistema PATH per includere la folder bin/protoc

GRPC per Java: compilazione protobuf



- Il compilatore per la generazioni delle classi GRPC va invocato specificando
  - I **path di output** per la generazione delle classi per la gestione di stub, canali, e messaggi (**--java_out=** e **--grpc-java_out=**)
  - Il path per il **plugin protoc** da usare per la compilazione (scaricare la giusta versione https://repo.maven.apache.org/maven2/io/grpc/protoc-gen-grpc-java/).
    Per esempio, per la versione **1.73.0 (quella da utilizzare)** del plugin:
    - Linux: https://repo.maven.apache.org/maven2/io/grpc/protoc-gen-grpc-java/1.73.0/protoc-gen-grpc-java-1.73.0-linux-x86_64.exe
    - MacOSX: https://repo.maven.apache.org/maven2/io/grpc/protoc-gen-grpc-java/1.73.0/protoc-gen-grpc-java-1.73.0-osx-x86_64.exe
    - Windows: https://repo.maven.apache.org/maven2/io/grpc/protoc-gen-grpc-java/1.73.0/protoc-gen-grpc-java-1.73.0-windows-x86_64.exe
    - NOTA BENE: una volta scaricato il file, dare i permessi di esecuzione! (e.g., in Linux *chmod +x NOMEFILE*)

- $ protoc **--java_out**=. **--grpc-java_out**=. **--plugin**=protoc-gen-grpc-java=/PATH/TO/protoc-gen-grpc-java-1.9.1-osx-x86_64 nome_file.proto

- NOTA BENE: Per alcuni sistemi operativi (ad es. MacOS) vengono sollevate delle eccezioni di sicurezza. È quindi necessario fare in modo (attraverso le impostazioni del sistema operativo in uso) che il plugin sia considerato sicuro.

GRPC per Java: librerie e dipendenze



- Per la creazione di un progetto Java che usi GRPC bisognerà includere le seguenti librerie JAVA nel classpath. Utilizzare versioni compatibili tra loro!
  - Dovendo includere più jar files utilizzare `-cp /path/folder/jar/files/*` seguito dal separatore
- Le seguenti librerie e versioni sono state testate correttamente per il funzionamento degli esercizi del corso (tali librerie sono disponibili sul repo del corso sottoforma di jar files)
  - `failureaccess-1.0`
  - `grpc-api-1.73.0`
  - `grpc-core-1.73.0`
  - `grpc-netty-shaded-1.73.0`
  - `grpc-protobuf-1.73.0`
  - `grpc-protobuf-lite-1.73.0`
  - `grpc-stub-1.73.0`
  - `guava-33.4.8-jre`
  - `perfmark-api-0.26.0`
  - `protobuf-java-4.31.1`

GRPC e API Java



- I concetti di gRPC come ***channel***, ***stub***, e **messaggi Protobuf** sono gli stessi descritti in Python
- Tuttavia Java è più verboso nell'utilizzo delle API grpc, utilizza costruttori ed eccezioni *checked*
- Sia Java che Python supportano chiamate RPC bloccanti (sincrone) e non bloccanti (asincrone)

GRPC e API Java



- Per l'implementazione di applicazioni Java che utilizzino GRPC è necessario comunque
  - **Protobuf**
    - È necessario introdurre la sola riga **`option java_multiple_files = true;`** dopo quella **`syntax`**
    - Non obbligatoria, ma permette una migliore organizzazione delle classi generate da protoc
    - Ricordarsi di impostare appropriatamente il campo **`package`**
      - In Java non è ignorato ma viene utilizzato per determinare il percorso di package dove verrà generato il codice, a meno che non viene specificata l'option `option java_package`
      - Deve corrispondere allo stesso valore di package utilizzato per la generazione di applicazioni Python
  - **Classe/i client-side**
    - Creare un **channel**
    - Istanziare uno **stub** che utilizzi tale canale per le chiamate RPC
    - **Invocare** le chiamate RPC e **attendere le risposte**
  - **Classe/i server-side**
    - Istanziare un **servente**
    - Creare un **server GRPC** e fare il **bind** con il **servente**
    - **Avviare** il server e mettersi in **attesa** di servire richieste RPC
    - Gestire le richieste e **inviare le risposte**
  - **Classe/i di implementazione delle funzioni RPC**

Client-side GRPC APIs in Java



Concetto	Java gRPC API	Note e comparazione con GRPC Python
Import modulo gRPC	import io.grpc.*	Java utilizza importazioni esplicite dal pacchetto io.grpc. Python ha un modulo grpc più semplice e unificato.
Classe Stub	MyServiceGrpc.MyServiceBlockingStub / MyServiceStub	Java genera più tipologie stub (bloccanti, asincroni, *future*). Python fornisce un singolo stub, più semplice ma meno esplicito.
Channel	Grpc.newChannelBuilder(target, InsecureChannelCredentials.create()).build()	Java usa ManagedChannel creato tramite il pattern *builder*. Python chiama semplicemente grpc.insecure_channel(...).
Costruire un messaggio di richiesta	MyRequest.newBuilder().setField(...).build()	Java usa il pattern builder immutabile per i messaggi protobuf. Python usa direttamente gli argomenti delle parole chiave.
Chiamata a un metodo (bloccante)	blockingStub.myMethod(request)	Stessa semantica di stub.MyMethod(request) di Python, ma Java richiede un tipo di stub esplicito.
Chiamata a un metodo (async)	asyncStub.myMethod(request, responseObserver)	Java usa StreamObserver (pattern observer) per callback asincrone. Python usa .future() o asyncio (più pulito per casi semplici).
Gestione della risposta	response.getField()	Java usa metodi getter; Python usa l'accesso diretto agli attributi (response.field).
Gestione degli errori	catch (StatusRuntimeException e)	Java usa eccezioni checked e try-catch strutturati. Python usa try/except grpc.RpcError.
Terminazione e cleanup del canale	channel.shutdownNow().awaitTermination(timeout, unit)	Java impone la chiusura e cleanup manuale del canale. Python: channel.close() è più semplice, ma meno configurabile.

Server-side GRPC APIs in Java



Concetto	Java gRPC API	Note e comparazione con GRPC Python
Istanziare un server	Grpc.newServerBuilderForPort(port, credentials). Le credentials sono di tipo ServerCredentials che viene estesa da InsecureServerCredentials oppure TlsServerCredentials	Java utilizza un modello di costruttore; Python utilizza un modello di istanziazione diretta.
Aggiungere un servente	.addService(new MyServiceImpl())	Java usa una classe che estende *ImplBase, Python usa una funzione helper per registrare il servizio.
Avviare il server	.build().start()	Esplicito in entrambi. Java usa il concatenamento con il builder; Python è più imperativo.
Fermare il server	server.shutdown() oppure server.shutdownNow()	Entrambi supportano uno shutdown graceful; Java consente un controllo più preciso con lo shutdown asincrono.
Implementare un servizio	Estende MyServiceGrpc.MyServiceImplBase e fa override dei metodi	Entrambi seguono principi orientati agli oggetti per definire la logica del servizio.
Gestire risposta unary	Uso di responseObserver.onNext(response) e responseObserver.onCompleted()	Java utilizza callback tramite StreamObserver; Python utilizza lo statement return
Gestire risposta streaming	Uso di chiamete multiple a onNext(), seguite da onCompleted()	Java usa lo stile push con StreamObserver; Python usa i generator per lo streaming server-side
Threading / Execution	Un ExecutorService custom (e.g., utilizzando FixedThreadPool) passato attraverso il metodo .executor(...)	Entrambi consentono il controllo dell'uso dei thread. Java utilizza una configurazione che permette più controllo tramite la classe ExecutorService.
Gestione della terminazione	Registra hook di shutdown alla JVM (Runtime.getRuntime().addShutdownHook(...))	Java ha bisogno di hook specifici per arrestare il server; Python usa tipicamente gestori di segnali o gestori di contesto.
Gestione degli errori	Uso di responseObserver.onError(StatusException)	Entrambi espongono codici di stato gRPC ed eccezioni per propagare gli errori.

Esempio: Hello world Java RPC (server-side)



```
public class HelloWorldServer {

 private Server server;
 private void start() throws IOException {
  /* The port on which the server should run */
  int port = 0;
  ExecutorService executor = Executors.newFixedThreadPool(2);
  server = Grpc.newServerBuilderForPort(port, InsecureServerCredentials.create())
              .executor(executor)
              .addService(new GreeterImpl())
              .build()
              .start();
  System.out.println("Server started, listening on " + server.getPort());

  . . .

      try {
         server.awaitTermination(); //wait for termination
      } catch (InterruptedException e) {
      // HANDLE. . .
      }
}
. . .
```

Esempio: Hello world Java RPC (server-side)



```
public class HelloWorldServer {

 private Server server;
 private void start() throws IOException {
  /* The port on which the server should run */
  int port = 0;
  ExecutorService executor = Executors.newFixedThreadPool(2);
  ser
        ServerBuilder<?> serverBuilder = Grpc.newServerBuilderForPort(port, InsecureServerCredentials.create());
        serverBuilder.executor(executor);
        serverBuilder.addService(new GreeterImpl());
        server = serverBuilder.build();
        server.start();

  System.out.println("Server started, listening on " + server.getPort());

  . . .

      try {
         server.awaitTermination(); //wait for termination
      } catch (InterruptedException e) {
      // HANDLE. . .
      }
}
. . .
```

Esempio: Hello world Java RPC (server-side)



```
public class HelloWorldServer {
  . . .
  /**
  * Main launches the server from the command line.
  */
  public static void main(String[] args) throws IOException, InterruptedException {
            final HelloWorldServer server = new HelloWorldServer();
            server.start(); //start routine will await for termination
  }

}
```

Esempio: Hello world Java RPC (client-side)



```
public class HelloWorldClient {

 private final GreeterGrpc.GreeterBlockingStub blockingStub;

 /** Construct client for accessing HelloWorld server using the existing channel. */
 public HelloWorldClient(Channel channel) {
  // 'channel' here is a Channel, not a ManagedChannel, so it is not this code's responsibility to
  // shut it down.

  // Passing Channels to code makes code easier to test and makes it easier to reuse Channels.
  blockingStub = GreeterGrpc.newBlockingStub(channel);
 }

 /** Say hello to server. */
 public void greet(String name) {
  System.out.println("Will try to greet " + name + " ...");
  HelloRequest request = HelloRequest.newBuilder().setName(name).build();
  HelloReply response;
  try {
    response = blockingStub.sayHello(request);
  } catch (StatusRuntimeException e) {
    System.out.println("RPC failed: {0} " + e.getStatus());
    return;
  }
  System.out.println("Greeting: " + response.getMessage());
 }
 . . .
```

Esempio: Hello world Java RPC (client-side)



```
public class HelloWorldClient {

  . . .
  /**
  * Greet server. If provided, the first element of {@code args} is the name to use in the
  * greeting. The second argument is the target server.
  */
  public static void main(String[] args) throws Exception {
    String user = "world";
    String target = "localhost:"+args[0];
    ManagedChannel channel = Grpc.newChannelBuilder(target, InsecureChannelCredentials.create()).build();
    try {
      HelloWorldClient client = new HelloWorldClient(channel);
      client.greet(user);
    } finally {
      channel.shutdownNow().awaitTermination(5, TimeUnit.SECONDS);
    }
  }
}
```

Esempio: Hello world Java RPC (impl)



```
public class GreeterImpl extends GreeterGrpc.GreeterImplBase {

        @Override
        public void sayHello(HelloRequest req, StreamObserver<HelloReply> responseObserver) {
         System.out.println("[SERVER JAVA GRPC] Invoked sayHello");

         HelloReply reply = HelloReply.newBuilder().setMessage("Hello " + req.getName()).build();

         System.out.println("[SERVER JAVA GRPC] Set reply and send...");

         responseObserver.onNext(reply);
         responseObserver.onCompleted();
        }
}
```